

Farhan
1603084

Database Management System

Farhan
1603084

** We will be considering these schemes throughout the book! ~

Books -

<u>ID</u>	Name	Genre	Writer-ID	Price	Rating
-----------	------	-------	-----------	-------	--------

↳ Sometimes Book-name
Writers -

<u>Writer-ID</u>	Name	No. of Books
------------------	------	--------------

↳ Sometimes writer-name

Publishers

<u>Publisher-ID</u>	Name	No. of Books
---------------------	------	--------------

Database:-

Database is a collection of interrelated data.

Database Management System (DBMS):-

A DBMS is a collection of interrelated data and a set of programs to access them. The collection of data contains information relevant to an enterprise.

* Primary goal → To ^{provide a way to} store and retrieve ^{large amount of} database information in an efficient and convenient way.

* Safety → The database system must ensure safety of the information stored, despite system crashes and unauthorized access.

Database-System Applications:-

* Universities:-

For student information, course registrations, and grades.

* Airlines:-

For reservations and schedule information. Airlines were among the first to use databases in a geographically distributed manner.

* Telecommunication:-

For keeping records of calls made, generating monthly bills, maintaining balances on prepaid calling cards and storing information about the communication networks.

(More in book)

Traditional file-processing system:-

Before DBMSs were introduced, organizations stored information in such systems.

The system stores permanent records in various files and (.txt, .dat), and it needs different application programs to extract records from, and add records to, the appropriate files.

Disadvantages of traditional file-processing system:-

(1) Data redundancy and inconsistency:-

- Since different programmers created the files and the application programs for over a long period, the various files are likely to have different structures and the programs may be written in different languages. Moreover, the same information may be duplicated in several files. This redundancy leads to higher storage and access cost.

- It may lead to data inconsistency. That is, the various copies of same data may no longer agree. For example - a changed student address may be reflected in the English department records, but not elsewhere in the system.

(2) Difficulty in accessing data:-

More responsive and convenient data-retrieval systems are required for general use. But, traditional file-processing environment do not allow needed data to be retrieved in a convenient and efficient manner.

For example - Suppose that one of the university clerks needs to find out the names of all students who live in a particular area. He has two choices - (i) extract the needed information manually, or (ii) ask a programmer to write a necessary program.

Both alternatives are unsatisfactory, because, after several periods, new criterion might come and new program need to be written.

(3) Data isolation:-

- Data might be scattered in various files.
- Data might be stored in different formats, so we need to write new programs, which is difficult.
- There might be no relationship among files.

(4) Integrity problems:-

The data values stored in a database must satisfy certain amounts of types of consistency constrains. Suppose the university maintains an account for each department and the account balance must never fall below zero. The developers enforce these ~~pro~~ constrains by adding appropriate code.

However, when ^{new} constrains are added, it is difficult to change the programs. The problem is compounded ~~with~~ when constrains involve several data items from different files.

(5) Atomicity problems:-

Consider a program to transfer 500/- from account of A to account of B. If a system failure occurs at the time of execution of program, it is possible that the 500/- was removed

from account of A, but was not credited to account of B. Clearly, it is essential to database consistency that both the credits and debit occur, or neither occur. That is, the fund transfer must be atomic - it must happen entirely or nothing at all. It is difficult to ensure atomicity in a traditional file-processing system.

(6) Security problem:-

Not every user need to access all data. For example, in a university, a payroll personnel need not access academic records. But, since application programs are added to the file processing system in an ad-hoc manner, enforcing such security constrain is difficult.

▣ Instances of DB:-

The collection of database information stored in the database at a particular moment is called an instance of the database.

Instance of a DB changes frequently.

▣ Schema of DB:-

The overall design of the DB is called the database schema. It does not change frequently.

Data models:-

(1) Relational model:-

- The relationship model uses a collection of tables to represent both data and the relationships among those data.
- Tables are also known as relations.
- Each table has unique name
- Each table has a set of columns called attributes.

(2) Entity-relationship model:-

- The ER model uses a collection of basic objects, called 'entities', and relationships among these objects.
- An entity is a thing or an object that is in the real world and it is distinguishable from other objects.

(3) Object-based data model:-

- It can be seen as extending the ER model with notions of encapsulation, methods and object identity.
- The object-relational data model combines features of the object oriented data model and relational data model.

(4) Semistructured data model:-

- Extensible Markup Language (XML) is an example of semistructured data model.

Database languages:-

(1) Data definition language (DDL):-

Contains the commands used to define database schema

Ex-

- To create a table
- To drop a table
- To create column
- To drop column

(2) Data manipulation language (DML):-

Contains the commands used to manipulate data in the database.

Ex-

- Insertion of data
- Deletion of data
- Retrieval of data
- Modification of data

Database Administrator:-

The person who have the central control of both the data and the programs that access the data is called a database administrator (DBA).

Functions of a DBA:-

(1) Schema definition:- DBA creates the original database schema by executing a set of data definition statements in DDL.

(2) Storage structure and access method definition

(3) Schema and physical-organization modification:-

The DBA carries out changes to the schema and physical organization to reflect the changing needs of the organization or to improve performance.

(4) Granting of authorization of data access:- The DBA can regulate which part of the DB can be accessed by which user.

(5) Routine maintenance:-

- Periodically backing up DB either onto tapes or onto remote servers, to prevent loss of data in case of disasters.

- Ensuring that enough free disk space is available for normal operations and upgrading disk space if required.

- Monitoring jobs running on the DB and ensuring that performance is not degraded by very expensive tasks submitted by users.

Chapter-2

Relational Database Model:-

* A Relational database consists of a collection of tables, each of which is assigned a unique name.

Table $\longrightarrow$ Relation

Row $\longrightarrow$ Tuple

Column $\longrightarrow$ Attribute

* For each attribute of a relation, there is a set of permitted values called domain of that attribute.

* We require that, for all relations r , the domains of all attributes of r be atomic. A domain is atomic, if the elements of the domain are considered indivisible units.

- For example - phone number is considered unatomic, because it can be divided into country code, operator code and the number.

+ 8801711234568

- But, if we consider the whole number as a single undivisible unit, then it is atomic.

- Null $\longrightarrow$ unknown / doesn't exist

Keys:-

We must have a way to specify how tuples within a given relation are distinguished. This is expressed in terms of their attributes. The values of the attribute values of a tuple must be such that they can uniquely identify the tuple. In other words, no two tuples in a relation are allowed to have exactly the same values for all attributes.

(1) Super key:-

A superkey is a set of one or more attributes that allow us to identify uniquely a tuple in a relation.

A superkey may contain extraneous attributes. For example - the combination of ID and name is a super key. If K is a superkey, then so is any superset of K .

(2) Candidate key:-

We are often interested in superkeys for which no proper subset is a superkey. Such minimal superkeys are called candidate keys.

There might be multiple candidate key for a single relation.

(3) Primary key:-

Primary key is a candidate key chosen by the DB designer, as the principal means of identifying tuples within a relation.

The primary key should be chosen such that its attribute values are never, or ^{very} rarely changed.

There is only one primary key possible.

(4) Foreign key:-

A relation may contain primary key of another relation as an attribute. It is called foreign key.

Example:- A book DB attributes -

<u>ID</u>	Name	Writer	Year	Rating
-----------	------	--------	------	--------

↳ P.K.

<u>Set of attribute</u>		<u>Super Key</u>		<u>Candidate key</u>		<u>Primary Key</u>
{ID, Name}	-	✓	-	X	-	X
{ID}	-	✓	-	✓	-	✓
{Name}	-	X	-	X	-	X
{Name, Year}	-	✓	-	✓	-	X
{Name, Year, Rating}	-	✓	-	X	-	X
{Year}	-	X	-	X	-	X

Relation Schema:-

It is the overall design of the relation. It consists of a list of attributes and their corresponding attributes domains.

Relation Instance:-

The data in a relation for instance time.

Chapter-3

Basic Types:-

* char(n): A fixed length character string with user specified length n.

Ex - name char(10) → "Sakib"-----
→ Adds spaces

* Varchar(n): A variable length character string with user specified length n.

Ex - name varchar(10) → "shakib"
→ no addition of spaces

* int: An integer

* smallint: A small integer

* numeric(p,d): A fixed point number with user specified precision. The number consists of p digits and d of the p digits are right of the decimal point.

* real, double precision: Floating point and double precision floating point numbers with machine dependent precision.

* float(n): A floating point number, with precision of at least n digits.

SQL-parts:-

* DDL (Data Definition Language):-

Provides commands for

- Defining relational schema
- Adding columns/new relations
- Dropping columns/~~new~~ relations

* DML (Data Manipulation Language):-

Provides commands for-

- Inserting new tuple
- Deleting tuples
- Modifying tuples

Table creation:-

create database Book-info; → DB creation

create table Books

(ID int,
Name varchar(100),
Writer varchar(100),
Year varchar(5),
Rating Numeric(3,1)
Writer-ID int,
primary key (ID),

foreign key (Writer-ID) references (writers));

Insertion:-

insert into Books

values (1, 'Things Fall Apart', 'Chinua Achebe', 1958,
3.6, 1);

Primary Key :- $\rightarrow$ values unique
 $\rightarrow$ not null

Primary Key,
Foreign key
 $\downarrow$
not null
constraints

* Method - 1 :-

```
create table Books  
( ID int,  
  :  
  primary key (ID) );
```

* Method - 2 :-

```
create table Books  
( ID int primary key,  $\rightarrow$  not possible for  
  :  
  );  
  )
```

* Method - 3 :-

creating an attribute and making it primary key -

alter table Books add ID int primary key;

* It is a problem when we immediately create attribute and before inserting values, next line we make it primary key.

```
alter table Books add ID int;  $\rightarrow$  All ID null  
alter table Books add primary key (ID);
```

$\rightarrow$ (X)

Solution -

```
alter table Books add ID int unique  
                        ^  
                        not null;
```


Deletion:-

* Deleting all tuples, but keeping the table -

delete from Books;

* Delete the table -

drop table Books;

Add/Delete Attributes:-

* Add new column -

alter table Books

add Prices numeric(4,1);

↗ All the values will be null

* Delete column -

alter table Books

drop Prices;

Queries:-

Books

↗ "Good read" ratings are on 5.0, not 10.0

ID	Name	Genre	Rating	Writer-ID Director-ID	Price
1.	Things fall apart	Novel	3.6	1	16.5
2.	Salah-ad-deen	History	4.0	3	30.0
3.	The art of war	Biography	4.0	2	10.0
4.	What world lost with the fall of Muslims	History	4.5	4	25.5
5.	Rumi	Poetry	4.3	null	null

Writers

<u>Writer-ID</u>	<u>Name</u>	<u>No.-of- books</u>
1.	Chinua Achebe	32
2.	Ali Sun Tzu	1
3.	Dr. Ali Sallabi	100
4.	Ali Hadwi	50
5.	Tamim Shahriar	10

* Find Name, Genre, Rating of all Books-

select Name, Genre, Rating
from Books;

* Find Name of all ~~movies~~ books-

select Name
from Books;

* Select distinct genre for all books-

select distinct Genre
from Books;

* Select all columns from table-

select *
from Books;

* Find Name, Genre and Price of all books in tk-

select Name, Genre, Price * 83.33
from Books;

Name	Genre	Price * 83.33
------	-------	---------------

↳ Temporary relation

* Find all books name, rating and price of all the books of 'History' genre-

select Name, Rating, Price
from Books
where Genre = 'History';

* Find name, rating and price of all books of history genre and containing rating greater than 3.75-

select Name, Rating, Price
from Books
where Genre = 'History' and Rating > 3.75;

**	<u>Comparison Operators</u> > greater < less = equal >= greater or equal <= less or equal <> not equal	<u>Logic connections</u> and or not
----	--	--

A typical SQL Query has the form-

```
select A1, A2, A3, ..., An
from r1, r2, ..., rm
where p;
```

* Find the name, genre, rating and writer for all books-

```
select Books.Name, Genre, Rating, Writer.Name
from Books, Writers
where Books.Writer_ID = Writers.Writer_ID;
```

* Find the name, genre, rating and writer name of all history genre books-

```
select Book-Name, Genre, Rating, Writer-Name
from Books, Writers
where Books.writer-ID = Writers.Writer-ID
and Genre = 'History';
```

It is a good practice to name same type of attributes in different table by different name. We had Name's both Book and Writers table. We changed them to Book-Name and Writer-Name

Update:-

* Update book name 'Salah ad-deen' to 'Salah ad-deen Ayubi' -

Update Books

set Name = 'Salah ad-deen Ayubi'

where ID = 1;

Rename operation:-

select Books.Name, Genre, Writers.Name as 'Writer'
from Books as B, Writers as W
where B.Writer-ID = W.Writer-ID;



Name	Genre	Writer
------	-------	--------

String Operations:-

- (Underscore) → single and any character.

% (Percent) → 0 or more characters.

* Find all book name starts with 'T'

select *

from ~~database~~ Books

where Name like 'T%';

* Find all book name second character 'h'

select *

from Books

where Name like '-h%';

* All writers with surname 'Nadwi' -

```
select *  
from Writers  
where Name like '%Nadwi';
```

* Book name that has '2' or 'two' anywhere -

```
select *  
from Books  
where Name like '%2%'  
or Name like '%two%';
```

* Writer name has only two characters -

```
select *  
from Writers  
where name like ' _ _';
```

□ Ordering:-

* Display book names and ratings sorted by book names -

```
select * Name, Rating  
from Books  
order by Name asc;      ↗ if ascending  
                          desc      ↘ if descending
```


* Display name and ratings are sorted by names and if names are equal, then by rating-

```
select Name, Rating
from Books
order by Name asc, Rating desc;
```

* select writers who does not have 'a' at second place-

```
select *
from Writers
where Name not like '_a%';
```

Between:-

* Select books where rating $4 \leq r \leq 4.5$ -

```
select *
from Books
where Rating >= 4.0
and Rating <= 4.5;
```

or

```
select *
from Books
where Rating between 4.0 and 4.5;
```

Union
intersect
except } Set operations

Union-operation:-

Math

ID	Name	Age	CGPA
1	A	-	-
2	B	-	-
3	C	-	-

Physics

ID	Name	Age	CGPA
2	B	-	-
3	C	-	-
4	D	-	-

* Find ID, Name and Ages of all students who is either enrolling Math or Physics-

(Select ID, Name, Age
from Math)

Union

(select ID, Name, Age
from Physics);

* If the attributes were M-Name and P-Name-

(select ID, M-Name as Name, Age
from Math)

Union

(select ID, P-Name as Name, Age
from Physics);

→ (2) 3 3 3 3 3 !!!

Null values:-

* Arithmetic operation on null is null-

Price * 8.33 → null
null

* Result of comparison operator on null is null-

Rating > 5 → null
null

* Logical operation on null-

_____ and _____
null true → null
null false → false

_____ or _____
null true → true
null false → null

not _____
null → null

Aggregate Functions:-

Functions that take a collection of values as input and return a single value.

SQL offers 5 built in aggregate fⁿs.

- Average: avg()
- Minimum: min()
- Maximum: max()
- Total: sum()
- Count: count()

sum
avg { Must be numeric

max
min
~~total~~
count { Can also operate on string

* Find the average price of the books:-

```
select avg(Price)
from Books;
```

* Find avg price of History genre-

```
select avg(Price)
from Books
where Genre = 'History';
```

* Find the total number of books in DB-

```
select count(ID)
from Books;
```

Or ↘

```
select count(*)
from Books;
```

* Find the number of book categories-

```
select count(distinct Genre)
from Books;
```

▣ Aggregation with Grouping:-

* find avg. price of each book category-

```
select avg(Price)
from Books
group by Genre;
```

Having clause:-

* Find the book categories with avg. price greater than 10.0 \$.

```
select avg(Price)
  from Books
  group by Genre
  having avg(Price) > 10.0;
```

where $\rightarrow$ condition for each tuple

having $\rightarrow$ condition for each group

* The meaning of a query containing aggregation, group by or having clauses is defined by the following sequence of operations -

(1) The from clause is first evaluated to get the relation

(2) If a where clause is present, the predicate in the where clause is applied on the result relation of the from clause.

(3) Tuples satisfying the where predicate are then placed into groups by the group by clause. If group by clause is absent, the entire set of tuples satisfying the where predicate is treated as being in one group.

(4) The having clause is applied to each group.

(5) The select clause uses the remaining groups to generate tuples of the result of the query, applying the aggregate functions to get a single result from each group.

4 Nesting subqueries:-

* Find all the books with genre History, Novel, Biography-

```
select *  
from Books  
where Genre in  
('History', 'Novel', 'Biography');
```

* Find Name, Rating, Price of all Books of Dr. Ali Sallabi and Ali Nadwi-

```
select Name, Rating, Price  
from Books  
where Writer_ID in  
(select Writer_ID  
from Writers  
where Writer Name in  
('Dr Ali Sallabi', 'Ali Nadwi'));
```

* Find the name and ~~rate~~ genre of the book with highest rating:-

```
select Name, Genre, max(Rating) as maxRating  
from Books;
```

X → The first name of the table will be shown
" " genre " "
" max rating " " " " " "


```
select Name, Genre  
from Books  
where Rating =  
    (select max(Rating)  
     from Books,  
    );
```

foreign key can be null, but not any value that does not exist in referenced table

Chapter-4

⊗ Find name, rating of the books with corresponding writer name ~~being matched~~ :-

Select Name, Rating, Writer-Name
from Books join Writers
using (writer-ID);

⊗ Find Name, Rating, Writer-Name of all the books. If writer name and other info are unknown, show null.

↳ Not possible with natural join
we use outer join.

⌘ select Name, Rating, Writer-Name
from Books left outer join Writers
using (writer-ID);

↳ Left table will be displayed full and then will match with right table with left table.

1. Books join writers

↳ Display where no null occurs

2. Books left^{outer} join writers

↳ Includes^{all} output of ①

- Books those do not match/null writer-ID,
the Book info will be as in it and the
writer fields will be null.

3. Books right outer join writers

↳ Include all output of ①
- Reverse of ②

* Full outer

4. Books full outer join Writers
↳ All will be included

Views:-

- SQL allows a "virtual relation" to be defined by a query, and the relation conceptually contains the result of the query. The virtual relation is not precomputed and stored, but instead is computed and by executing the query whenever the virtual relation is executed.
- Any such relation that is not part of the logical model, but is made visible to a user as a virtual relation, is called a view.

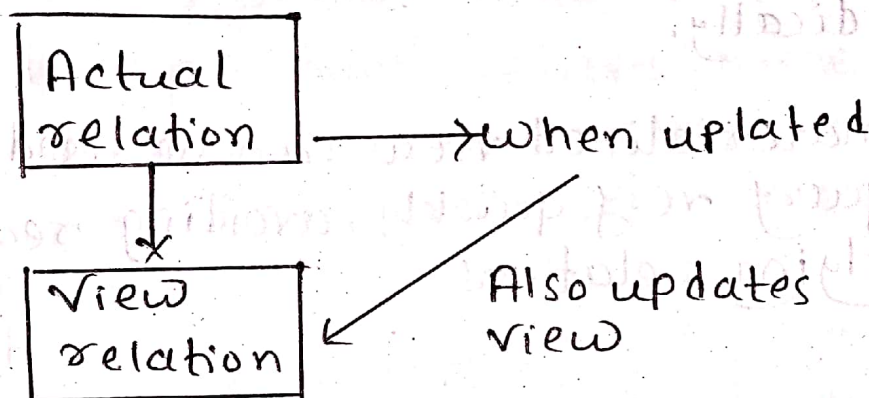
create view viewName as
<query>;

- The database system stores the query expression associated with the view relation.

* Example -

create view IMPbookinfo as
(select book-name, genre, rating
from book);

select genre, count(*), avg(rating)
from IMPbookinfo
group by genre;



Materialized view:-

- Certain database system allow view relations to be stored, but they make sure that, if the actual relations used in the view definition change, the view is kept up to date. Such views are called - materialized views.

- Store view as a relation
- Non-standard

- The process of keeping materialized view up-to-date is called materialized view maintenance/view maintenance.
- View maintenance can be done immediately when any of the relations on which the view is defined is updated.
- Some database system perform view maintenance lazily, when the view is accessed.
- Some systems update materialized views periodically.
- The materialized view can be used to answer the query very quickly, avoiding reading large underlying relations.

Update of a View:-

* Problem-1:-

- The difficulty is that a modification to the ~~database expressed in terms of a view~~ must be translated to a modification to the actual relations in the logical model of that database.
- insert into IMPbookinfo
values ("Eat that frog!", "non-fiction", 8.5)
→ ID, writer-ID, price missing

There are two reasonable approaches to dealing with this insertion -

- (1) Reject the insertion and return an error message to the user. "non-fiction"
- (2) Insert a tuple (null, "Eat that frog!", 8.5, null, null) into the Books relation.

* Problem-2: -

- Create view book-writer-info as
(select Books.name, writers.name, rating
from Books join writers
using (writer-ID)
);

insert into book-writer-info
values ("Eat that frog", "Brian Tracy", 8.5)

- The only possible method of inserting tuples into Books and writers is to insert (null, "Eat that frog", null, 8.5, null, null) to Books and (null, "Brian Tracy", null) to Writers. This update does not have the desired effect.

→ view update is not permitted where the view is derived from join of relations.

⊗ Problem-3:~

— create view history-books as

(select *

from Books

where Genre = 'History'

);

Suppose the user tries to insert (6, "Eat that frog!", "non-fiction", 8.5, null, 10.0) into the history-books view, it will be inserted into the Books relation, but will not appear in the view, because of the filter, Genre = 'History'.

→ with check option can be added at the view's query to reject these update.

⊗ Conditions when the view is updatable:~

(1) The from clause has only one database relation.

(2) The select clause contains only attribute names of the relation.

→ No expression,
No aggregates,
No distinct

- (3) The query does not have a group by or having clause.
- (4) Any attribute not listed in the select clause can be set to null, if it does not have a not null constrain or not a primary key.

Integrity constraints:-

- Integrity constraints ensure that changes made to the database by authorized users do not result in a loss of data consistency. Thus integrity constraints guard against accidental damage to the database.
- o A writer name can't be null → not null
- o Not two writers can have same ID → primary key
- o writer-ID in Books relation must match with the writer-ID in Writers relation → foreign key
- o ~~Dept~~ Department name must be unique → unique
- o Book price must be greater than 0.00 → check
- When such a command is executed, the system first ensures that the relation satisfied the specific constrain. If it does, the constrain is added to the relation, if not, the command is rejected.

Not null:-

- o Book-name varchar(50) not null
- o rating numeric(3,1) not null

↳ SQL prohibits null values in the primary key automatic. we do not need to declare explicitly.

Unique:-

- o unique($A_1, A_2, \dots, A_m$)

— The unique specification says that the attributes $A_1, A_2, \dots, A_m$ form a candidate key. No two tuples in the relation can be equal ~~to~~ on all the listed attributes.

— However, candidate key attributes are permitted to be null unless they have explicitly been declared to be not null.

check():-

— when applied to a relation declaration, the clause check(P) specifies a predicate P that must be satisfied by every tuple in the relation.

P $\hookrightarrow$ Domain of an attribute
 $\hookrightarrow$ A particular set of values

-Ex-

create table Book

(book-ID int primary key,
book-name varchar(50) not null,
genre varchar(20),
check (genre in ('History', 'Biography')));

- Currently none of the widely used database products allows the predicate to be a subquery.

Referential integrity:-

Foreign key

↳ Referential integrity constrain

-Ex-

foreign key (writer-ID) references writers (writer-ID)

Default:-

↳ Assigns a default value.

-Ex-

create table Books

(book-ID int primary key,
book-name varchar(50) not null,
price int default 0,
);

☐ Date and Time types:-

(1) date → 'YYYY-MM-DD'

(2) time → 'HH-MM-SS'

(3) timestamp → 'YYYY-MM-DD HH-MM-SS'

↳ Can have fractional part

'2001-04-25 10:29:01.45'

⊗ extract from date and time:-

extract (field from AttributeName)

↓
year

month

day

hour

minute

second

timezone-hour

timezone-minute

- Ex -

create table books

(id int primary key,

release-date Date,

release-time Time

);

insert into books

values (1, "2019-01-01", "12:00:00") ;

extract

select id, extract (year from release-date) as r-year
from books;

- current-date → returns current date
- current-time → current time + timezone
- localtime → current local time
- current-timestamp → date + time + timezone

Authorization:-

- Authorization to read data
- " " insert " }
- " " update " } Privilege
- " " delete " }

→ give authorization

⊗ grant <privilege list>

↙

take away

on <relation name or view name>
to <user/role list>

- Ex-

grant select, insert
on Books
to Farhan, Salman;

⊗ grant revoke privilege (attribute list)
on <relation or view name>
to <user/role list>;

- Ex -

grant update (book-name)
on Books
to salman;

- User name public refers to all current
and ~~previous~~ future users of the system.